A MINI PROJECT REPORT

On

THE ROUTING ROOM – HYBRID NLP-BASED SUPPORT TICKET CLASSIFICATION, RETRIEVAL AND INTELLIGENT ROUTING SYSTEM

Submitted in partial fulfillment of the requirement of

University of Mumbai for the Course

Natural Language Processing

In

Computer Engineering (VII SEM)

Submitted By

Somil Dedhiya

Uday Dutta

Sumeet Gupta

Subject Incharge

Prof. Suchita Dange

Department Of Computer Engineering

A. P. SHAH INSTITUTE OF TECHNOLOGY

THANE – 400 615

UNIVERSITY OF MUMBAI

Academic Year 2026 – 27

Abstract

The Routing Room is an explainable NLP system for triaging HackerRank support tickets. Given a ticket subject and issue description, it classifies the request type with linguistic rules, predicts the product area with a supervised TF-IDF logistic-regression classifier, and retrieves relevant support articles through hybrid retrieval that fuses TF-IDF cosine similarity with sentence-embedding similarity from all-MiniLM-L6-v2. The ten best fused candidates are re-scored by the cross-encoder ms-marco-MiniLM-L-6-v2. An explicit routing layer then decides whether the ticket can be answered from documentation or must be escalated to a human, based on missing detail, privileged actions, high-risk terms, outage signals, weak retrieval evidence, or ambiguous top results. Replies are extractive: sentences are selected from the top-ranked article rather than generated.

The system is implemented as a FastAPI backend with a static HTML/CSS/JavaScript interface offering an existing-ticket board, an interactive triage form, ranked recommendations with score components, and a decision trace. An optional Groq-hosted language model can rephrase the final result without influencing any decision. On the original 437-article corpus, five-fold cross-validation of the product-area classifier gave 0.858 accuracy and 0.844 macro F1 for TF-IDF logistic regression, against 0.332 accuracy for a keyword baseline. In a 20-query retrieval smoke test, TF-IDF retrieval followed by cross-encoder re-ranking reached recall@1 of 0.55 and nDCG@10 of 0.645, against 0.40 and 0.541 for lexical retrieval alone. The report also documents the limitations of this evaluation and of the current implementation.

Keywords: *Natural Language Processing, Support Ticket Triage, TF-IDF, Sentence Embeddings, Cross-Encoder Re-ranking, Hybrid Retrieval, Text Classification, Explainable AI, Escalation Routing.*

List of Figures

Fig. No.	Title of Figure	Page No.
Fig 3.1	Proposed system architecture	3
Fig 3.2	Implemented NLP triage pipeline	4
Fig 3.3	Interactive triage sequence	5
Fig 4.1	Retrieval alpha sweep (nDCG@10 against α)	9
Fig 4.2	Confusion matrix of the selected product-area classifier	9
Fig 4.3	Home page of the application	10
Fig 4.4	Existing Dataset board with summary cards and predictions	11
Fig 4.5	Interactive triage result for a ticket that is replied to	11
Fig 4.6	Interactive triage result for a ticket that is escalated	11

List of Tables

Table No.	Title of Table	Page No.
Table 2.1	Literature survey summary	2
Table 3.1	Routing conditions, in evaluation order	6
Table 3.3	Project modules and their responsibilities	7
Table 3.4	Software packages and pretrained models	7
Table 4.1	Ticket input fields of POST /api/triage	8
Table 4.2	Retrieval results on the 20-query smoke run	8
Table 4.3	Product-area classification, five-fold cross-validation	9

Chapter 1: Introduction

1.1 Fundamentals

Support teams receive a continuous stream of tickets written in free, informal language. Handling one normally requires a person to read the text, decide its type, find the relevant help article, judge whether that article answers the question, and decide whether it is safe to answer without a human, each step is an NLP task: classification, retrieval and ranking, and decision support.

This project uses rule-based classification for request type, a supervised TF-IDF classifier for product area, and hybrid retrieval that fuses TF-IDF cosine similarity with MiniLM sentence-embedding similarity, re-ranked by a cross-encoder. The reply shown to the user is extractive, built only from sentences already present in the top-ranked article, so nothing is invented and the decision to answer or escalate is made by explicit rules kept separate from retrieval, so that a sensitive request is sent to a human even when a matching article exists. Every decision carries a trace of the scores and rules that produced it.

1.2 Objectives

- Design an NLP pipeline that classifies a HackerRank support ticket by request type and product area.
- Retrieve relevant support articles by fusing TF-IDF and MiniLM embedding similarity, and re-rank candidates with a cross-encoder.
- Separate automatic replies from escalations through explicit, inspectable routing rules.
- Ground replies in the retrieved article through extractive answers and expose a decision trace.
- Evaluate the classifier and retriever with reproducible datasets and state the limits of that evaluation.

1.3 Scope

The implemented scope is a single-domain, English-language application for HackerRank support tickets, built over the Markdown knowledge base under data/hackerrank/. Corpora for two other companies, used in an earlier multi-domain version, are archived and excluded from inference. The system uses pretrained models for embeddings and re-ranking without fine-tuning any Transformer, does not generate free-form answers, and never modifies the original ticket files, tickets submitted through the browser are kept only in local storage. Authentication, server-side persistence and multilingual input are outside the scope of this work.

Chapter 2: Literature Survey

Routing a support ticket sits at the intersection of text classification and information retrieval: a useful system must identify what the user is asking, locate relevant documentation, rank several candidates, and answer while staying faithful to the available evidence. Table 2.1 summarizes eight works that directly informed the design of the Routing Room, spanning classical term weighting, contextual language models, dense retrieval and neural re-ranking.

Table 2.1: Literature survey summary

SN	Technique	Author, Year	Advantage	Disadvantage
1	TF-IDF term weighting	Salton & Buckley, 1988	Transparent, cheap, exact term matching	Misses paraphrases; ignores word order
2	BERT (pre-trained encoders)	Devlin et al., 2019	Strong contextual representation, transfers across tasks	Heavy to run; best results need fine-tuning
3	Sentence-BERT (bi-encoder)	Reimers & Gurevych, 2019	Fast cosine search; tolerant to paraphrase	Depends on encoder quality; may blur exact terms
4	BERT passage re-ranking (cross-encoder)	Nogueira & Cho, 2019	Accurate joint query–passage scoring	Cost grows with candidates; short lists only
5	Dense passage retrieval	Karpukhin et al., 2020	Finds passages with little word overlap	Needs a good encoder/training data
6	ColBERT late interaction	Khattab & Zaharia, 2020	Token-level matching, offline doc encoding	Larger index; more complex implementation
7	MiniLM distillation	Wang et al., 2020	Small, fast models that run on a CPU	Lower capacity than full-size models
8	Learning to rank	Liu, 2009	Formalizes ranking; optimizes ranking metrics	Needs labelled ranking data

The literature points to complementary strengths: lexical methods are transparent and exact, dense methods handle paraphrase, and cross-encoders give the most accurate final ordering but are affordable only on a short list. The Routing Room combines the first two signals and applies a cross-encoder to a small candidate pool, suiting a support corpus where transparency and source-grounded output matter.

Chapter 3: Project Implementation

3.1 Overview

The Routing Room is organized in four layers: a static HTML/CSS/JS presentation layer served by FastAPI; an API layer (app.py) exposing JSON endpoints and holding one shared triage agent; an engine layer (code/) implementing the NLP pipeline; and a data layer holding the Markdown knowledge base, cached embeddings, the persisted classifier and ticket CSVs (Fig 3.1). A transparent in-memory index is used instead of a vector database since the knowledge base holds only a few hundred articles.

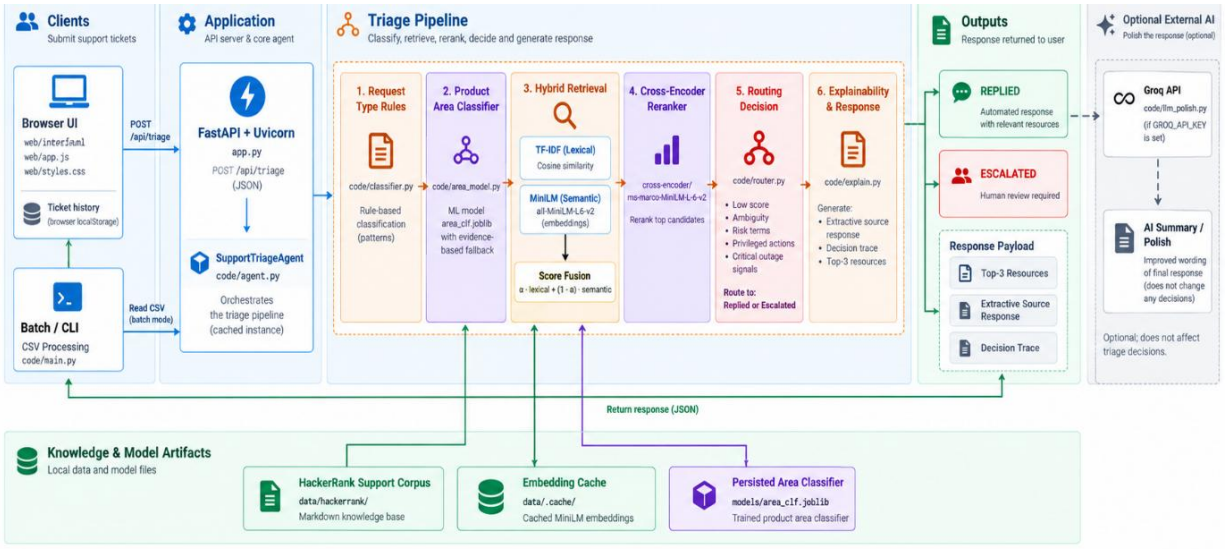


Fig 3.1: Proposed system architecture

3.1.1 Existing Methodology

Manual triage relies on an agent reading each ticket, tagging it and searching the help centre. Automated alternatives each have gaps: keyword rules are auditable but need constant maintenance; pure lexical search finds exact terms but misses paraphrases; pure dense retrieval recovers paraphrases but can lose exact feature names; and generative assistants write fluent answers but need strong grounding controls. Most systems also conflate “which article matches” with “should a machine answer at all”, a ticket asking to raise a candidate’s score may match an article well yet should still go to a person.

3.1.2 Proposed Methodology

Every ticket passes through seven inspectable stages (Fig 3.2):

- Rule-based request-type classification (invalid, feature request, bug, product issue).
- Query construction: the ticket text is split into clauses; the two longest become queries.
- Hybrid retrieval: TF-IDF cosine and MiniLM embedding similarity fused with weight α.
- Cross-encoder re-ranking of the ten best fused candidates.
- Supervised product-area classification.
- Rule-based routing between replied and escalated.
- Extractive reply from the top article, a decision trace, and an optional LLM summary that cannot change any decision.

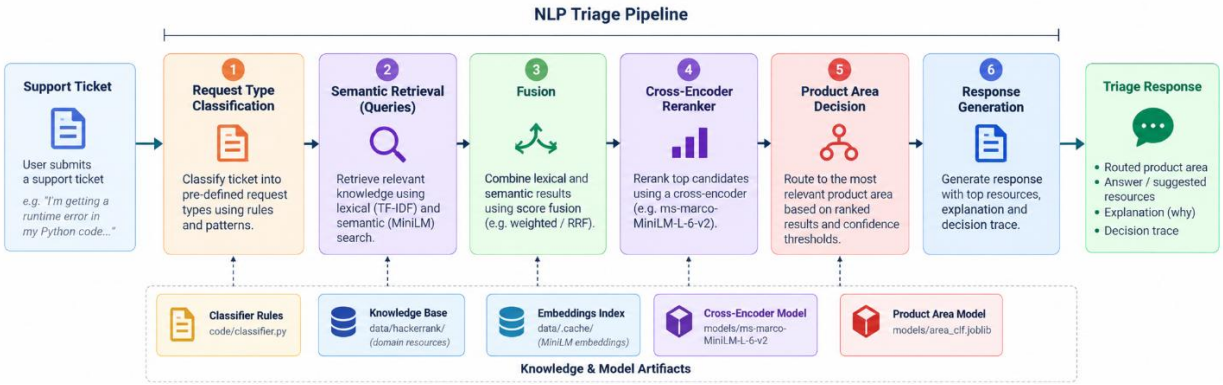


Fig 3.2: Implemented NLP triage pipeline

3.2 Implementation Details

The central class, SupportTriageAgent, builds a Ticket object, classifies request type, builds query clauses, retrieves and merges evidence, predicts the product area, applies routing rules, and returns a TriageDetails object serialized to JSON (Fig 3.3).

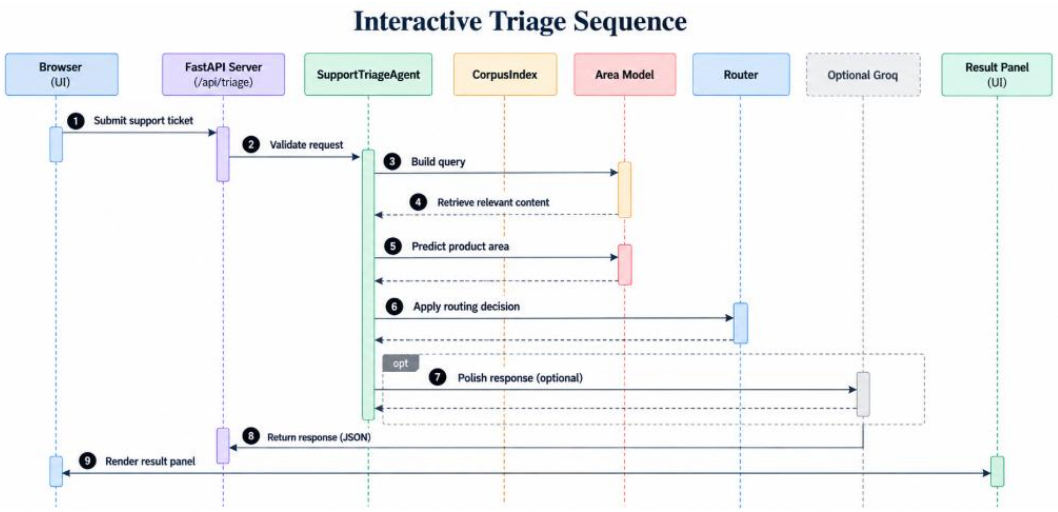


Fig 3.3: Interactive triage sequence

3.2.1 Methodology

- Query construction – subject and issue are split on punctuation/conjunctions into clauses; the two longest distinct clauses become search queries.
- Lexical retrieval – each Markdown article is tokenized, lower-cased and stop-word filtered; TF–IDF cosine similarity is computed over an index of 545 articles and 39,359 distinct terms, with per-token attribution recorded.
- Semantic retrieval – each article's title plus first 1,800 characters is encoded by all-MiniLM-L6-v2 into a cached unit-length embedding; the query is encoded the same way.
- Score fusion – $S(d,q) = \alpha \cdot L/\max L + (1-\alpha) \cdot \max(0,E)$, with default $\alpha = 0.4$.
- Cross-encoder re-ranking – the ten best fused candidates are re-scored by ms-marco-MiniLM-L-6-v2; the top 3 per query are merged and ordered by fused score.
- Request-type rules – ordered regex rules mark a ticket invalid, feature_request or bug; anything else is product_issue.
- Product-area classifier – a TF-IDF (uni+bigram) logistic-regression pipeline trained on each article's title and first 200 tokens; falls back to evidence-based inference if the model is unavailable.
- Routing – router.py applies the ordered conditions of Table 3.1, plus a confidence tag (high/medium/low) from the top fused score.
- Extractive response – the three sentences (≥35 characters) with the highest ticket-overlap are returned from the top article; escalated tickets get a fixed human-review message.

- Decision trace – records the request-type rule, product area and probability, confidence, routing reason, score components and score gap.
- Optional LLM polish – when a Groq API key is set, the already-decided outcome is rephrased by gpt-oss-120b/20b; the layer cannot alter any decision.
- Web application – the FastAPI backend and static frontend (home page, Existing Dataset board, Interactive Triage workspace) keep browser-submitted tickets only in local storage, so the original CSV is never modified.

Table 3.1: Routing conditions, in evaluation order

#	Condition (evaluated in order)	Outcome
1	Request type is invalid	Not escalated (auto-reply)
2	Fewer than 3 content tokens after stop-word removal	Escalated – insufficient detail
3	Privileged action (change/raise a score; unlock/delete an account) or a high-risk term (fraud, security vulnerability, account takeover)	Escalated – privileged / high-risk
4	Bug ticket containing “down” or “none of”	Escalated – outage signal
5	No retrieval evidence, or top fused score below 0.30	Escalated – weak evidence
6	Gap between top two scores below 0.03	Escalated – ambiguous
7	None of the above	Replied – clear recommendation

3.2.2 Packages and Dataset

The active knowledge base holds 545 Markdown articles across eight product areas (with three small sections merged into “other”); the stored evaluation uses the earlier 437-article snapshot, predating 108 articles added later. The inherited ticket file support_tickets.csv has 29 tickets — 14 HackerRank, 7 Claude, 6 Visa and 2 unlabelled — all of which the current engine processes as HackerRank tickets. Table 3.3 lists the project modules and Table 3.4 the main packages and pretrained models the engine depends on. The API itself exposes a small set of endpoints — GET / for the interface, GET /api/health, GET /api/corpus-stats, GET /api/dataset and POST /api/triage — with the triage agent created once and cached so that models load only on the first request. The frontend additionally offers a light/dark theme toggle, and all text returned by the server is HTML-escaped before being inserted into the page, so the interface cannot be used to inject markup.

Table 3.3: Project modules and their responsibilities

Module	Responsibility
app.py	FastAPI server, routes, dataset loading, agent cache and API serialization
code/agent.py	Orchestration: ticket, request type, queries, retrieval, area, routing, response, trace
code/corpus.py	Indexing, TF-IDF retrieval, MiniLM embeddings and cache, fusion, cross-encoder re-ranking
code/classifier.py, area_model.py	Request-type rules; persisted product-area classifier and evidence-based fallback
code/router.py	Escalation rules and the confidence tag
code/explain.py	Extractive response and decision-trace construction
code/llm_polish.py	Optional Groq summary layer, downstream of all decisions
web/, eval/	Static frontend (HTML, CSS, JS); dataset builder, evaluations and plots

Table 3.4: Software packages and pretrained models

Package / Model	Role in the project
FastAPI, Uvicorn, Pydantic	Web framework, ASGI server and request validation
scikit-learn, joblib	TF-IDF vectorizer, logistic regression, cross-validation, model persistence
NumPy	Embedding matrix and dot-product scoring
sentence-transformers	Loads the MiniLM encoder and the cross-encoder
all-MiniLM-L6-v2	Sentence encoder for semantic retrieval
ms-marco-MiniLM-L-6-v2	Cross-encoder for re-ranking
Groq API (gpt-oss-120b/20b)	Optional downstream summary layer

Chapter 4: Project Inputs and Outputs

4.1 Input Details

A ticket consists of a required issue description and an optional subject; a company field is accepted but ignored, since the engine is fixed to HackerRank (Table 4.1). Knowledge-base inputs are the Markdown articles under data/hackerrank/, each labelled by its directory. The Existing Dataset view re-runs all 29 tickets in support_tickets.csv through the same pipeline without modifying the file.

Table 4.1: Ticket input fields of POST /api/triage

Field	Constraint	Description
subject	Optional, ≤ 300 characters	Short title; joined with the issue text before processing.
issue	Required, 3 to 5,000 characters	Free-text description of the problem.
company	Optional, ≤ 80 characters	Accepted but ignored; the active domain is HackerRank.

4.2 Evaluation Parameters and Results

Retrieval is evaluated with Recall@k (k = 1, 3, 5), MRR@10 and nDCG@10; classification with accuracy, per-class precision/recall/F1 and macro F1. The classifier uses five-fold stratified cross-validation (seed 42) over 437 article texts, comparing keyword rules, TF-IDF logistic regression and MiniLM-embedding logistic regression. Retrieval is evaluated for lexical-only, semantic-only and hybrid fusion at several α values, and the best hybrid run plus cross-encoder re-ranking, on a 20-query smoke test.

Table 4.2: Retrieval results on the 20-query smoke run

Run	α	R@1	R@5	nDCG@10
Lexical only	–	0.40	0.55	0.5411
Semantic only	–	0.35	0.45	0.4309
Hybrid (fusion)	0.6	0.30	0.65	0.4966
Hybrid (fusion)	1.0	0.40	0.55	0.5411
Hybrid + cross-encoder	1.0	0.55	0.70	0.6447

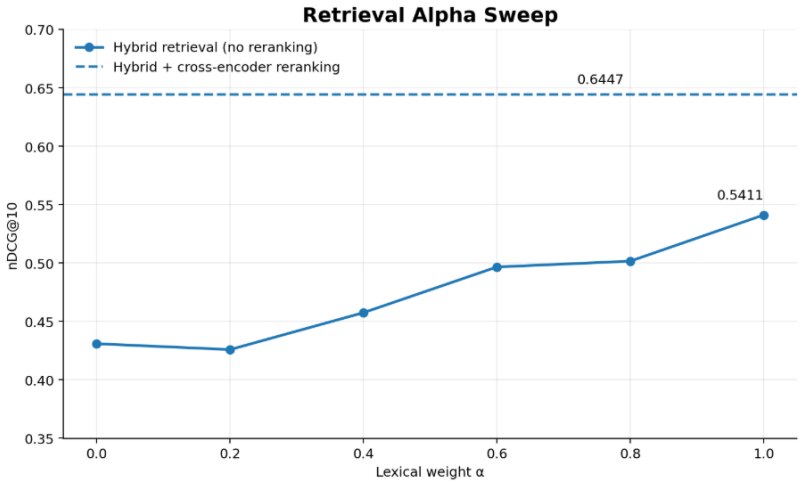


Fig 4.1: Retrieval alpha sweep (nDCG@10 against α)

Table 4.3: Product-area classification, five-fold cross-validation (437 articles)

Model	Accuracy	Macro F1
Keyword rules	0.3318	0.1911
MiniLM embeddings + logistic regression	0.7826	0.7479
TF-IDF + logistic regression (selected)	**0.8581**	**0.8440**

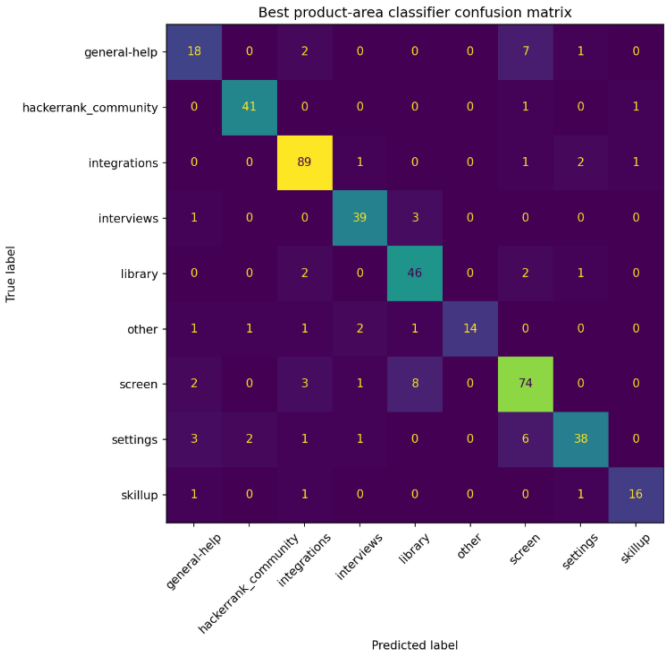


Fig 4.2: Confusion matrix of the selected product-area classifier

TF-IDF logistic regression clearly beats the keyword baseline (0.858 vs 0.332 accuracy) and MiniLM embeddings (0.783); the strongest class is hackerrank_community (F1 0.943), the weakest is general-help (0.667). On retrieval, no fused $\alpha < 1$ beat lexical-only retrieval on

nDCG@10, and adding the cross-encoder to lexical-only retrieval gave the largest gain — recall@1 rose from 0.40 to 0.55. With only 20 queries, differences smaller than about 0.10 should not be regarded as meaningful.

Limitations

- The 20-query smoke test covers only 10 articles from two small sections and is not representative of the whole corpus.
- Held-out text removal works for title queries only; lead queries overlap their source article, making those results optimistic.
- The best α was chosen on the same queries later reported, and the deployed agent orders the merged shortlist by fused score, not by the benchmarked re-ranked order.
- Stored results describe the 437-article snapshot, not the current 545-article corpus, and the classifier is validated on article text rather than real, user-written tickets.
- The outage rule matches “down” as a substring (mis-escalating words like “download”); the feature-request pattern can mistype tickets mentioning “support”; invalid tickets are answered rather than escalated; and extraction only sees an article's first 900 characters.

4.3 Output Details and Screenshots

For a replied ticket, the API returns up to three extracted sentences plus the source path; for an escalation it returns a fixed human-review message while the ranked resources are kept as context. The interface (Figs 4.3–4.6) shows a status badge, request type, confidence, a Best Match card with lexical/semantic/rerank scores, the source response, an optional AI summary, the decision trace and two further recommendations.

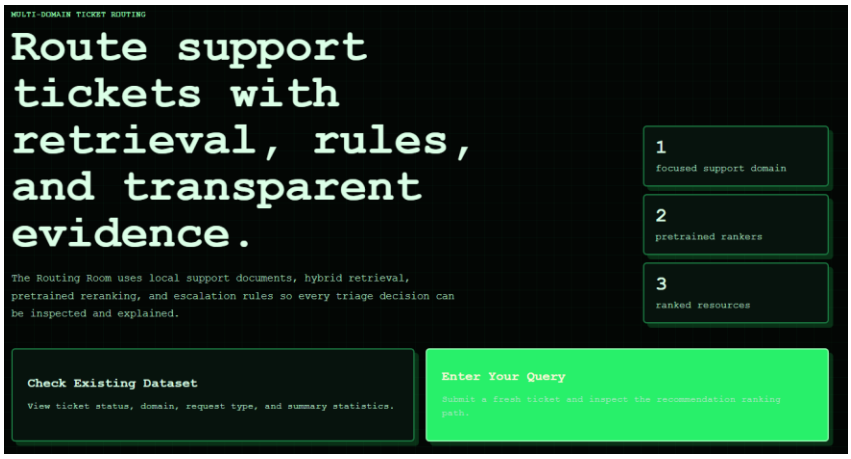


Fig 4.3: Home page of the application

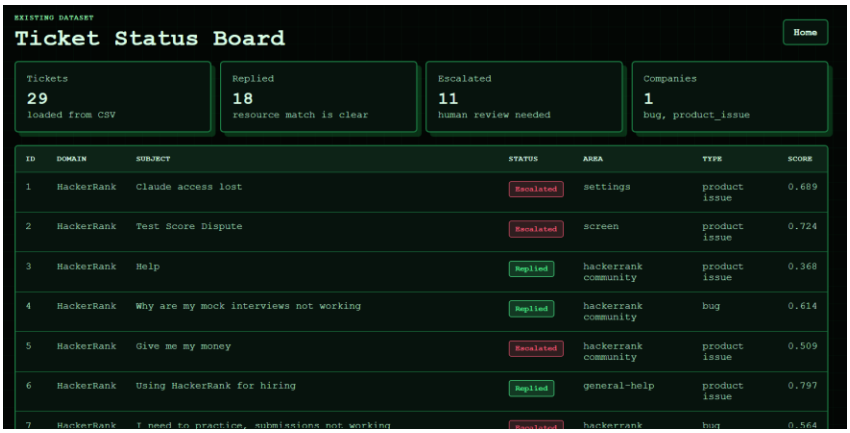


Fig 4.4: Existing Dataset board with summary cards and per-ticket predictions

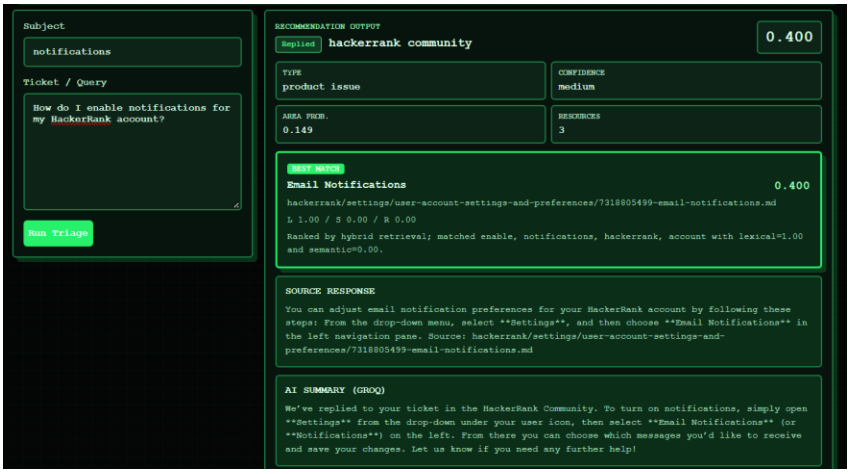


Fig 4.5: Interactive triage result for a ticket that is replied to

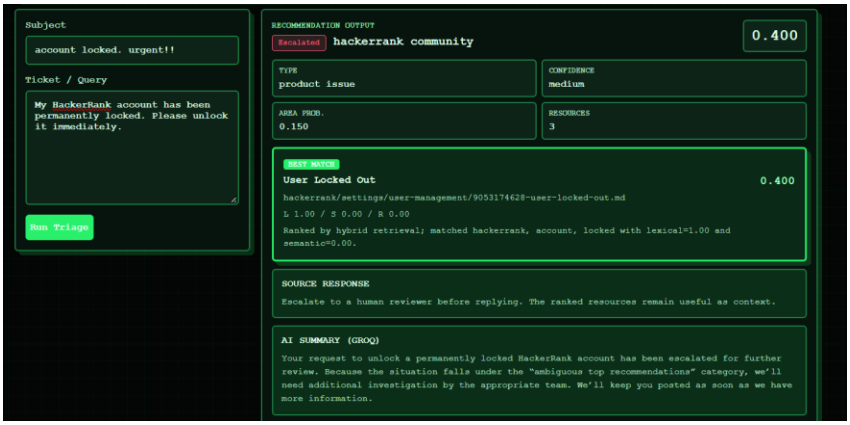


Fig 4.6: Interactive triage result for a ticket that is escalated

Chapter 5: Summary and Future Scope

5.1 Summary

The Routing Room implements an end-to-end NLP triage pipeline for HackerRank support tickets, combining rule-based request typing, supervised product-area classification, TF-IDF and MiniLM retrieval with score fusion, cross-encoder re-ranking, explicit escalation rules, extractive replies and a decision trace, exposed through a FastAPI backend and a static web interface. On the 437-article snapshot, the TF-IDF logistic-regression classifier reached 0.858 accuracy and 0.844 macro F1 under five-fold cross-validation; in a 20-query smoke run, cross-encoder re-ranking raised recall@1 from 0.40 to 0.55. These figures come from a small, partly optimistic evaluation and show that the pipeline works, not its production accuracy.

5.2 Future Scope

- Fix the observed issues: forward lexical attribution through fused search, order the merged shortlist by cross-encoder score, use whole-word outage matching, tighten the feature-request pattern, and give invalid inputs their own status.
- Strengthen retrieval evaluation with a stratified sample across all sections, corrected held-out text removal, separated tuning/test queries, and a BM25 baseline.
- Retrain the product-area classifier and repeat all evaluations on the current 545-article corpus.
- Collect labelled real tickets to evaluate request type, product area and escalation on user-written language.
- Calibrate routing thresholds on validation data and report escalation precision/recall.
- Restore multi-domain support and prepare for production with a reviewer feedback loop, caching, authentication and monitoring.

References

[1] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2023.

[2] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional Transformers for language understanding,” in Proc. NAACL-HLT, 2021, pp. 4171–4186.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2024, pp. 3982–3992.

[4] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 2023.

[5] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in Proc. EMNLP, 2020, pp. 6769–6781.

[6] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in Proc. SIGIR, 2020, pp. 39–48.

[7] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained Transformers,” in NeurIPS 33, 2020, pp. 5776–5788.

[8] T.-Y. Liu, “Learning to rank for information retrieval,” Foundations and Trends in Information Retrieval, vol. 3, no. 3, pp. 225–331, 2019.

[9] T. Nguyen et al., “MS MARCO: A human generated machine reading comprehension dataset,” arXiv:1611.09268, 2016.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” JMLR, vol. 12, pp. 2825–2830, 2011.

[11] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[12] Sentence-Transformers, “all-MiniLM-L6-v2,” model card, Hugging Face, 2026.

[13] Sentence-Transformers, “ms-marco-MiniLM-L-6-v2,” model card, Hugging Face, 2026.

[14] S. Ramírez, “FastAPI documentation,” fastapi.tiangolo.com, 2026.